

Multi-Asset Trend Following Strategy

Production-Ready Architecture v3.2

February 2026 | Implementation Blueprint with Critical Fixes

DOCUMENT CONTROL

Version	Date	Changes
1.0	Q4 2025	Initial architecture
2.0	Jan 2026	Human-in-the-loop, explainability
3.0	Feb 2026	Bulk API optimization, formula precision, script separation
3.1	Feb 2026	Critical fixes: Entry signals, circuit breakers, validation thresholds, charting
3.2	Feb 2026	Download modes simplified: 3 modes (initial/incremental/custom) with auto symbol discovery

EXECUTIVE SUMMARY

Purpose

Production-grade systematic trend-following strategy with:

- **Rule-based execution** (zero discretion, explicit formulas)
- **Human-in-the-loop** oversight (approval required)
- **Bulk data architecture** (EODHD bulk API for speed)
- **Modular script design** (independent execution)
- **Complete auditability** (decision trail)
- **Circuit breakers** (prevent catastrophic losses)

Target

- Account size: €10,000 → €100,000+
- Asset classes: Stocks (NYSE/NASDAQ, EU), ETFs (Global), Crypto (limited)
- Holding period: Medium to long-term (weeks to months)
- Rebalancing: Monthly (last trading day)
- Risk per position: 2% of account equity

Key Design Decisions

1. **NYSE + NASDAQ only** (no OTC, no complex filtering)
2. **Bulk API downloads** (all exchanges, post-filter in-memory)
3. **Separated data scripts** (EODHD and Yahoo independent)
4. **Formula precision** (every rule explicitly defined)
5. **Human approval gate** (system recommends, human executes)
6. **Circuit breakers** (automatic halt on extreme conditions)
7. **Strict validation** (explicit thresholds, no silent failures)

PART 1: DATA ARCHITECTURE

1.1 UNIVERSE DEFINITION

Eligible Instruments

United States Stocks

Exchanges: NYSE, NASDAQ only (no OTC, no BATS, no IEX)
Market Cap: ≥ \$500 million USD
Average Daily Volume (30-day): ≥ \$5 million USD
Minimum Price (30-day): ≥ \$4.50 USD
Listing History: ≥ 252 trading days
Currency: USD (converted to EUR)

European Stocks

Exchanges: XETRA (.DE), Euronext Paris (.PA), Euronext Amsterdam (.AS)
Market Cap: ≥ €200 million EUR
Average Daily Volume (30-day): ≥ €5 million EUR
Minimum Price (30-day): ≥ €5.00 EUR
Listing History: ≥ 252 trading days
Currency: EUR (native) or local (converted to EUR)

United Kingdom Stocks

Exchange: London Stock Exchange (.L)
Market Cap: ≥ £200 million GBP
Average Daily Volume (30-day): ≥ £5 million GBP
Minimum Price (30-day): ≥ £4.00 GBP
Listing History: ≥ 252 trading days
Currency: GBP (converted to EUR)

ETFs (All Regions)

Exchanges: NYSE, NASDAQ, XETRA, LSE, Euronext
Assets Under Management: ≥ €50 million
Average Daily Volume (30-day): ≥ €5 million
Minimum Price (30-day): ≥ €5.00
Listing History: ≥ 252 trading days
Exclusions: Leveraged ETFs (2×, 3×), Inverse ETFs

Cryptocurrencies

Exchange: Bitvavo (EUR pairs only)
Market Cap: ≥ €4 billion
24h Volume: ≥ €1 million
Age: ≥ 365 days
Maximum Allocation: 20% of portfolio
Supported: BTC, ETH only (for now)

Exclusions (Hard Rules)

- ✗ OTC Markets (excluded at source by using NYSE/NASDAQ exchanges)
- ✗ Penny stocks (price below minimum thresholds)
- ✗ Leveraged/inverse ETFs (distorted price action)
- ✗ Stocks with pending corporate actions (splits, mergers in next 30 days)
- ✗ Stocks with insufficient data history (<300 trading days)

1.2 DATA SOURCES & SCRIPTS

Script 1: EODHD Bulk Downloader

File: scripts/01_download_eodhd_bulk.py

Purpose: Download all market data using EODHD Bulk API with smart delta updates

Inputs:

- .env file with EODHD_API_KEY
- config/exchanges.json (exchange definitions)
- data_cache/metadata/last_update.json (for incremental mode)

Outputs:

- data_cache/raw_bulk/{exchange}/{date}.parquet (raw OHLCV)
- data_cache/metadata/last_update.json (last date per exchange)
- data_cache/metadata/bulk_download_log.json (audit trail)
- data_cache/metadata/new_symbols.jsonl (newly discovered symbols)
- data_cache/metadata/delisted_symbols.jsonl (delisted/suspended symbols)
- data_cache/corporate_actions/{exchange}_splits.parquet
- data_cache/corporate_actions/{exchange}_dividends.parquet

Execution Modes (3 total):

```
# MODE 1: INITIAL - Complete rebuild (run once at setup)
python scripts/01_download_eodhd_bulk.py --mode initial
# → Downloads 400 days for ALL symbols on all 6 exchanges
# → Creates baseline for future incremental updates
# → API calls: 2,412 | Time: 10-15 minutes
```

```
# MODE 2: INCREMENTAL - Smart delta load (production workhorse)
python scripts/01_download_eodhd_bulk.py --mode incremental
# → Auto-detects gap (last_date → today)
# → Downloads only missing dates
# → Discovers NEW symbols (IPOs, listings) automatically
# → Flags DELISTED symbols automatically
# → API calls: 6-144 (depending on gap) | Time: 30s - 4min
# → Self-healing (catches missed days automatically)

# MODE 3: CUSTOM - Specific date range (manual backfill)
python scripts/01_download_eodhd_bulk.py --from 2024-01-01 --to 2024-12-31
# → Downloads exact date range specified
# → Useful for gap filling or testing
# → API calls: varies | Time: varies
```

MODE 1: INITIAL (Complete Rebuild)

Use when:

- First time setup
- Cache corruption
- Disaster recovery

Logic:

```
def initial_mode():
    """
    Complete rebuild from scratch
    """
    # Step 1: Delete existing cache (with confirmation)
    if os.path.exists('data_cache/raw_bulk'):
        confirm = input("Delete existing cache? (yes/no): ")
        if confirm.lower() == 'yes':
            shutil.rmtree('data_cache/raw_bulk')

    # Step 2: Calculate 400 days back from today
    end_date = datetime.today()
    start_date = end_date - timedelta(days=400)

    # Step 3: Download all dates for all exchanges
    for exchange in EXCHANGES:
        logger.info(f"Downloading {exchange}: 400 days")

        for date in get_trading_days(start_date, end_date):
            # Bulk API returns ALL symbols on exchange for this date
            data = fetch_bulk_eod(exchange, date)
            save_to_parquet(f'data_cache/raw_bulk/{exchange}/{date}.parquet', data)

    # Step 4: Download corporate actions (5 years)
    for exchange in EXCHANGES:
        splits = fetch_bulk_splits(exchange, years=5)
        dividends = fetch_bulk_dividends(exchange, years=5)
        save_corporate_actions(exchange, splits, dividends)

    # Step 5: Initialize metadata
    initialize_last_update(end_date)
```

MODE 2: INCREMENTAL (Smart Delta Load)

Use when:

- Daily operations (automated cron)
- Weekly catch-up (after weekend)
- Monthly rebalancing (ensure data current)
- Any regular update

Features:

-  **Auto gap detection** - calculates (last_date → today)
-  **New symbol discovery** - detects IPOs, new listings
-  **Delisting detection** - flags symbols that disappear
-  **Self-healing** - catches missed days automatically
-  **Validation** - checks for gaps in cache

Logic:

```
def incremental_mode():
    """
    Smart delta load with auto-discovery
    """
```

```

# Step 1: Load last update date per exchange
last_updates = load_last_updates()
# Example: {'NYSE': '2026-02-07', 'NASDAQ': '2026-02-07', ...}

today = datetime.today().date()

for exchange in EXCHANGES:
    last_date = last_updates.get(exchange)

    if last_date is None:
        logger.error(f"{exchange}: No baseline. Run --mode initial first.")
        continue

    # Step 2: Calculate date range (last_date + 1 → today)
    start_date = last_date + timedelta(days=1)
    end_date = today

    # Step 3: Get only trading days in range
    trading_days = get_trading_days(start_date, end_date)

    if len(trading_days) == 0:
        logger.info(f"{exchange}: Already up to date")
        continue

    logger.info(f"{exchange}: Downloading {len(trading_days)} days ({start_date} → {end_date})")

    # Step 4: Track symbols for new/delisted detection
    previous_symbols = get_symbols_from_cache(exchange, last_date)

    for date in trading_days:
        # Bulk API call (returns ALL symbols on exchange for this date)
        data = fetch_bulk_eod(exchange, date)
        current_symbols = set(data['symbol'].unique())

        # Step 5: Detect NEW symbols (IPOs, new listings)
        new_symbols = current_symbols - previous_symbols
        if len(new_symbols) > 0:
            logger.info(f"{exchange} {date}: {len(new_symbols)} new symbols detected")
            log_new_symbols(exchange, new_symbols, date)

        # Step 6: Detect DELISTED symbols (on last date only)
        if date == trading_days[-1]:
            potentially_delisted = previous_symbols - current_symbols
            if len(potentially_delisted) > 0:
                logger.warning(f"{exchange} {date}: {len(potentially_delisted)} symbols missing")
                log_delisted_symbols(exchange, potentially_delisted, last_date)

        # Save data
        save_to_parquet(f'data_cache/raw_bulk/{exchange}/{date}.parquet', data)
        previous_symbols = current_symbols

    # Step 7: Update corporate actions for delta period
    append_corporate_actions(exchange, start_date, end_date)

    # Step 8: Update last update metadata
    update_last_update(exchange, end_date)

# Step 9: Validate no gaps in last 300 days
validate_no_gaps(lookback_days=300)

```

New Symbol Discovery:

```

def log_new_symbols(exchange: str, symbols: set, date: date):
    """
    Log newly discovered symbols to new_symbols.jsonl
    """
    with open('data_cache/metadata/new_symbols.jsonl', 'a') as f:
        for symbol in symbols:
            entry = {
                'exchange': exchange,
                'symbol': symbol,
                'first_seen_date': date.isoformat(),
                'reason': 'new_listing',
                'timestamp': datetime.now().isoformat()
            }
            f.write(json.dumps(entry) + '\n')

    logger.info(f"NEW SYMBOLS: {' '.join(symbols)}")

```

Delisting Detection:

```
def log_delisted_symbols(exchange: str, symbols: set, last_date: date):
    """
    Log potentially delisted symbols to delisted_symbols.jsonl
    """
    with open('data_cache/metadata/delisted_symbols.jsonl', 'a') as f:
        for symbol in symbols:
            entry = {
                'exchange': exchange,
                'symbol': symbol,
                'last_seen_date': last_date.isoformat(),
                'reason': 'delisted_or_suspended',
                'timestamp': datetime.now().isoformat()
            }
            f.write(json.dumps(entry) + '\n')

    logger.warning(f"POTENTIALLY DELISTED: {' '.join(symbols)}")
```

MODE 3: CUSTOM (Specific Date Range)

Use when:

- Filling specific gaps
- Reprocessing corrupted periods
- Testing with subsets

Logic:

```
def custom_mode(from_date: str, to_date: str):
    """
    Download specific date range
    """
    start_date = datetime.strptime(from_date, '%Y-%m-%d').date()
    end_date = datetime.strptime(to_date, '%Y-%m-%d').date()

    trading_days = get_trading_days(start_date, end_date)

    logger.info(f"Custom download: {len(trading_days)} trading days ({start_date} → {end_date})")

    for exchange in EXCHANGES:
        for date in trading_days:
            data = fetch_bulk_eod(exchange, date)
            save_to_parquet(f'data_cache/raw_bulk/{exchange}/{date}.parquet', data)

    # Note: Does NOT update metadata (to avoid interfering with incremental mode)
```

API Endpoints:

```
# OHLCV Data (bulk per exchange per date)
BASE_URL = "https://eodhistoricaldata.com/api/eod-bulk-last-day/{EXCHANGE}"
params = {
    'api_token': API_KEY,
    'date': 'YYYY-MM-DD',
    'fmt': 'json'
}

# Corporate Actions (bulk per exchange)
SPLITS_URL = "https://eodhistoricaldata.com/api/splits-bulk/{EXCHANGE}"
DIVIDENDS_URL = "https://eodhistoricaldata.com/api/dividends-bulk/{EXCHANGE}"
params = {
    'api_token': API_KEY,
    'from': 'YYYY-MM-DD',
    'to': 'YYYY-MM-DD',
    'fmt': 'json'
}
```

Exchange Configuration:

```
{
  "exchanges": {
    "NYSE": {
      "name": "New York Stock Exchange",
      "code": "NYSE",
      "country": "US",
      "currency": "USD",
      "trading_hours": "09:30-16:00 EST"
    },
    "NASDAQ": {
      "name": "NASDAQ Stock Market",
      "code": "NASDAQ",
      "country": "US",
```

```

    "currency": "USD",
    "trading_hours": "09:30-16:00 EST"
  },
  "XETRA": {
    "name": "Deutsche Börse XETRA",
    "code": "XETRA",
    "country": "DE",
    "currency": "EUR",
    "trading_hours": "09:00-17:30 CET"
  },
  "LSE": {
    "name": "London Stock Exchange",
    "code": "LSE",
    "country": "GB",
    "currency": "GBP",
    "trading_hours": "08:00-16:30 GMT"
  },
  "PA": {
    "name": "Euronext Paris",
    "code": "PA",
    "country": "FR",
    "currency": "EUR",
    "trading_hours": "09:00-17:30 CET"
  },
  "AS": {
    "name": "Euronext Amsterdam",
    "code": "AS",
    "country": "NL",
    "currency": "EUR",
    "trading_hours": "09:00-17:30 CET"
  }
}
}
}

```

Rate Limiting:

```

# EODHD limits: 5 requests per second for bulk API
RATE_LIMIT = {
    'max_calls_per_second': 5,
    'retry_on_429': True,
    'max_retries': 3,
    'backoff_multiplier': 2
}

```

Metadata Files:

```

data_cache/metadata/
├── last_update.json          # Last date downloaded per exchange
│   {
│       "NYSE": "2026-02-10",
│       "NASDAQ": "2026-02-10",
│       "XETRA": "2026-02-10",
│       "LSE": "2026-02-10",
│       "PA": "2026-02-10",
│       "AS": "2026-02-10"
│   }
├── new_symbols.jsonl         # Newly discovered symbols
│   {"exchange": "NYSE", "symbol": "ABNB.US", "first_seen_date": "2026-02-10", ...}
│   {"exchange": "NASDAQ", "symbol": "SNOW.US", "first_seen_date": "2026-02-10", ...}
├── delisted_symbols.jsonl    # Delisted/suspended symbols
│   {"exchange": "NYSE", "symbol": "XYZ.US", "last_seen_date": "2026-02-07", ...}
└── bulk_download_log.json    # Complete audit trail
    {
    "runs": [
        {
            "timestamp": "2026-02-10T06:00:15Z",
            "mode": "incremental",
            "exchanges": {
                "NYSE": {
                    "start_date": "2026-02-08",
                    "end_date": "2026-02-10",
                    "days_downloaded": 1,
                    "new_symbols": 2,
                    "delisted_symbols": 0,
                    "api_calls": 1
                }
            }
        }
    ]
    }
}

```

```
}
}
```

Performance Characteristics:

Mode	API Calls	Time	Use Frequency
INITIAL	2,412	10-15 min	Once (setup)
INCREMENTAL (1 day)	6	30-60 sec	Daily
INCREMENTAL (1 week)	30	1-2 min	Weekly
INCREMENTAL (1 month)	132	3-4 min	Monthly
CUSTOM (varies)	Varies	Varies	As needed

Operational Workflow:

```
# Initial Setup (Day 1)
python scripts/01_download_eodhd_bulk.py --mode initial

# Daily Operations (Automated Cron)
# Single command works for daily, weekly, or monthly gaps!
0 6 * * 1-5 /path/to/scripts/01_download_eodhd_bulk.py --mode incremental

# Monthly Rebalancing
python scripts/01_download_eodhd_bulk.py --mode incremental
# → Auto-detects gap and fills
# → Check new/delisted symbols before rebalancing

# Gap Repair (if needed)
python scripts/01_download_eodhd_bulk.py --from 2025-12-15 --to 2025-12-20
```

Download Statistics:

```
Initial Mode (400 days, 6 exchanges):
- API calls: 2,400 OHLCV + 12 corporate = 2,412 total
- Execution time: ~10-15 minutes (with rate limiting)
- Data volume: ~15-20 GB (uncompressed), ~3-5 GB (parquet compressed)

Incremental Mode (varies by gap):
- 1 day gap: 6 calls, 30-60 seconds
- 5 day gap: 30 calls, 1-2 minutes
- 22 day gap: 132 calls, 3-4 minutes
- Self-heals automatically regardless of gap size
```

Script 2: Yahoo Finance Fundamentals Downloader

File: scripts/02_download_yahoo_fundamentals.py

Purpose: Fetch company metadata (name, sector, market cap) from Yahoo Finance

Inputs:

- List of all symbols from EODHD bulk download
- data_cache/raw_bulk/ (to extract unique symbols)

Outputs:

- data_cache/fundamentals/company_info.json

Data Schema:

```
{
  "AAPL.US": {
    "symbol": "AAPL.US",
    "name": "Apple Inc.",
    "sector": "Technology",
    "industry": "Consumer Electronics",
    "market_cap": 2800000000000,
    "currency": "USD",
    "exchange": "NASDAQ",
    "country": "US",
    "employees": 164000,
    "fetched_at": "2026-02-09T10:30:00Z",
    "source": "yahoo"
  }
}
```

Execution:

```
# Fetch all symbols (first time)
python scripts/02_download_yahoo_fundamentals.py --mode full

# Update only missing symbols (incremental)
python scripts/02_download_yahoo_fundamentals.py --mode incremental

# Refresh all (ignore cache)
python scripts/02_download_yahoo_fundamentals.py --mode refresh

# Test with limited symbols
python scripts/02_download_yahoo_fundamentals.py --max-symbols 100
```

API Approach:

```
import yfinance as yf

def fetch_company_info(symbol: str) -> dict:
    """
    Fetch company fundamentals from Yahoo Finance

    Yahoo symbol mapping:
    AAPL.US -> AAPL
    BMW.DE -> BMW.DE
    VOD.L -> VOD.L
    """
    # Convert EODHD format to Yahoo format
    yahoo_symbol = convert_symbol(symbol)

    try:
        ticker = yf.Ticker(yahoo_symbol)
        info = ticker.info

        return {
            'symbol': symbol,
            'name': info.get('longName', ''),
            'sector': info.get('sector', ''),
            'industry': info.get('industry', ''),
            'market_cap': info.get('marketCap', 0),
            'currency': info.get('currency', ''),
            'exchange': info.get('exchange', ''),
            'country': info.get('country', ''),
            'employees': info.get('fullTimeEmployees', 0),
            'fetched_at': datetime.now().isoformat(),
            'source': 'yahoo'
        }
    except Exception as e:
        logger.error(f"Failed to fetch {symbol}: {e}")
        return None
```

Rate Limiting:

```
# Yahoo Finance informal limits
RATE_LIMIT = {
    'requests_per_second': 2, # Conservative to avoid blocks
    'retry_delay': 5,
    'max_retries': 3
}
```

Symbol Mapping:

```
def convert_symbol(eodhd_symbol: str) -> str:
    """
    Convert EODHD symbol format to Yahoo Finance format

    Rules:
    - .US suffix -> remove (AAPL.US -> AAPL for NYSE/NASDAQ)
    - .DE, .L, .PA, .AS -> keep (BMW.DE, VOD.L, etc.)
    - .CC suffix -> replace with native ticker (BTC.CC -> BTC-USD)
    """
    if '.US' in eodhd_symbol:
        return eodhd_symbol.replace('.US', '')
    elif '.CC' in eodhd_symbol:
        # Crypto: BTC.CC -> BTC-USD
        base = eodhd_symbol.replace('.CC', '')
        return f"{base}-USD"
    else:
```

```
# Keep as-is for EU/UK
return eodhd_symbol
```

Script 3: Data Consolidator & Validator

File: scripts/03_consolidate_validate_data.py

Purpose: Merge EODHD + Yahoo data, validate quality, create unified dataset

Inputs:

- data_cache/raw_bulk/{exchange}/{date}.parquet
- data_cache/fundamentals/company_info.json
- data_cache/corporate_actions/*.parquet

Outputs:

- data_cache/consolidated/{symbol}.parquet (per-symbol time series)
- data_cache/metadata/data_quality_report.json
- data_cache/metadata/validation_failures.csv

Execution:

```
# Full consolidation
python scripts/03_consolidate_validate_data.py --mode full

# Validate only (no file creation)
python scripts/03_consolidate_validate_data.py --mode validate-only

# Specific exchange
python scripts/03_consolidate_validate_data.py --exchange NYSE
```

Data Validation Rules (EXPLICIT THRESHOLDS):

```
class DataValidator:
    """
    Comprehensive data quality validation with explicit thresholds
    """

    # VALIDATION THRESHOLDS (from architecture recommendations)
    PRICE_JUMP_THRESHOLD = 0.25 # 25% single-day jump without corporate action
    MAX_MISSING_DATA_PCT = 0.05 # 5% missing bars allowed
    ATR_EXPLOSION_MULTIPLIER = 2.0 # ATR > 2x median = data error
    MAX_DATA_STALENESS_DAYS = 3 # Alert if cache > 3 days old

    def validate_completeness(self, df: pd.DataFrame) -> tuple:
        """
        Check for missing data

        Rules:
        - No gaps in dates (excluding weekends/holidays)
        - All OHLCV columns present
        - No NaN values in price columns
        - Missing data < 5% of expected trading days
        """
        # Get expected trading days (exclude weekends + known holidays)
        expected_days = get_trading_calendar(
            start=df.index.min(),
            end=df.index.max(),
            exchange=df['exchange'].iloc[0]
        )

        actual_days = set(df.index)
        missing_days = expected_days - actual_days

        # Allow up to 5% missing (for data feed issues)
        missing_pct = len(missing_days) / len(expected_days)

        if missing_pct > self.MAX_MISSING_DATA_PCT:
            return False, f"Missing {missing_pct:.1%} of trading days (threshold: {self.MAX_MISSING_DATA_PCT:.1%})"

        # Check for NaN values
        if df[['open', 'high', 'low', 'close', 'volume']].isna().any().any():
            return False, "NaN values in OHLCV columns"

        return True, "OK"

    def validate_price_consistency(self, df: pd.DataFrame) -> tuple:
        """
```

```

Check OHLC relationships

Rules:
- High >= Low (always)
- High >= Open, Close (always)
- Low <= Open, Close (always)
- Single-day returns < 25% (without corporate action)
"""
# OHLC consistency
violations = (
    (df['high'] < df['low']) |
    (df['high'] < df['open']) |
    (df['high'] < df['close']) |
    (df['low'] > df['open']) |
    (df['low'] > df['close'])
)

if violations.any():
    return False, f"OHLC violations: {violations.sum()} days"

# Price jump check (without known splits)
df['return'] = df['close'].pct_change()
extreme_returns = df[abs(df['return']) > self.PRICE_JUMP_THRESHOLD]

# Cross-check with corporate actions
splits = get_splits_for_symbol(df['symbol'].iloc[0])
split_dates = set(splits['date']) if len(splits) > 0 else set()

# Filter out legitimate split dates
unexplained_jumps = extreme_returns[
    ~extreme_returns.index.isin(split_dates)
]

if len(unexplained_jumps) > 0:
    return False, f"Unexplained price jumps >25%: {len(unexplained_jumps)} days"

return True, "OK"

def validate_volume(self, df: pd.DataFrame) -> tuple:
    """
    Check volume sanity

    Rules:
    - Volume > 0 for all days
    - No extreme volume spikes (>100x average without news)
    """
    if (df['volume'] == 0).any():
        zero_days = (df['volume'] == 0).sum()
        return False, f"Zero volume days: {zero_days}"

    avg_volume = df['volume'].rolling(30).mean()
    volume_spike = df['volume'] / avg_volume

    if (volume_spike > 100).any():
        spike_days = (volume_spike > 100).sum()
        return False, f"Extreme volume spike (>100x avg): {spike_days} days"

    return True, "OK"

def validate_atr(self, df: pd.DataFrame) -> tuple:
    """
    Check volatility sanity

    Rules:
    - ATR_20 < 2x median(ATR_20 over 200 days)
    """
    # Calculate TR
    df['tr'] = df.apply(
        lambda row: max(
            row['high'] - row['low'],
            abs(row['high'] - row.get('prev_close', row['close'])),
            abs(row['low'] - row.get('prev_close', row['close']))
        ),
        axis=1
    )
    df['prev_close'] = df['close'].shift(1)

    df['atr_20'] = df['tr'].rolling(20).mean()
    median_atr = df['atr_20'].rolling(200).median()

    if (df['atr_20'] > self.ATR_EXPLOSION_MULTIPLIER * median_atr).any():
        explosion_days = (df['atr_20'] > self.ATR_EXPLOSION_MULTIPLIER * median_atr).sum()
        return False, f"ATR explosion (>2x median): {explosion_days} days"

```

```

        return True, "OK"

def validate_data_freshness(self, df: pd.DataFrame) -> tuple:
    """
    Check if data is stale

    Rules:
    - Last data point must be within 3 trading days of today
    """
    last_date = df.index.max()
    today = datetime.now().date()
    days_stale = (today - last_date.date()).days

    # Account for weekends
    trading_days_stale = count_trading_days(last_date, today)

    if trading_days_stale > self.MAX_DATA_STALENESS_DAYS:
        return False, f"Data stale: {trading_days_stale} trading days old"

    return True, "OK"

```

Consolidation Logic:

```

def consolidate_symbol(symbol: str) -> pd.DataFrame:
    """
    Create unified time series for a symbol

    Process:
    1. Load all OHLCV bars from raw_bulk/
    2. Sort by date
    3. Remove duplicates (keep last)
    4. Adjust for splits/dividends
    5. Convert to EUR
    6. Add company metadata
    7. Validate quality
    8. Save to consolidated/
    """
    # 1. Load raw data
    raw_df = load_raw_bulk_data(symbol)

    # 2. Sort and deduplicate
    df = raw_df.sort_values('date').drop_duplicates(subset=['date'], keep='last')

    # 3. Adjust for corporate actions
    splits = load_splits(symbol)
    dividends = load_dividends(symbol)
    df = adjust_prices(df, splits, dividends)

    # 4. Convert to EUR
    fx_rates = load_fx_rates(df['currency'].iloc[0])
    df['close_eur'] = df['close'] / fx_rates
    df['volume_eur'] = df['volume'] * df['close_eur']

    # 5. Add metadata
    company_info = load_company_info(symbol)
    for col, val in company_info.items():
        df[col] = val

    # 6. Validate
    validator = DataValidator()
    checks = [
        validator.validate_completeness(df),
        validator.validate_price_consistency(df),
        validator.validate_volume(df),
        validator.validate_atr(df),
        validator.validate_data_freshness(df)
    ]

    df['data_quality'] = all([c[0] for c in checks])
    df['validation_issues'] = ', '.join([c[1] for c in checks if not c[0]])

    # 7. Save
    output_path = f"data_cache/consolidated/{symbol}.parquet"
    df.to_parquet(output_path, compression='snappy')

    return df

```

1.3 DATA SCHEMA

Consolidated Time Series Schema

File: data_cache/consolidated/{SYMBOL}.parquet

```
DataFrame Schema:
- date: datetime64[ns] (index)
- symbol: string (e.g., "AAPL.US")
- exchange: string (e.g., "NASDAQ")

# Price data (local currency)
- open: float64
- high: float64
- low: float64
- close: float64
- adjusted_close: float64 (split/dividend adjusted)
- volume: int64

# Price data (EUR)
- close_eur: float64
- volume_eur: float64

# Metadata
- currency: string (e.g., "USD")
- name: string (e.g., "Apple Inc.")
- sector: string (e.g., "Technology")
- industry: string
- market_cap: int64 (in USD or local currency)
- market_cap_eur: float64

# Quality flags
- data_quality: bool (True if all validations pass)
- validation_issues: string (comma-separated issues)
```

PART 2: STRATEGY RULES (PRECISE FORMULAS WITH FIXES)

2.1 UNIVERSE SCREENING

Script 4: Universe Screener

File: scripts/04_screen_universe.py

Purpose: Apply liquidity and fundamental filters to create qualified universe

Inputs:

- data_cache/consolidated/*.parquet
- data_cache/fundamentals/company_info.json
- config/filter_thresholds.json

Outputs:

- data_cache/qualified/qualified_symbols.json (list of passing symbols)
- data_cache/qualified/screening_report.csv (detailed results)

Execution:

```
python scripts/04_screen_universe.py --as-of-date 2026-01-31
```

Filter Thresholds:

```
{
  "NYSE": {
    "min_price_usd": 4.50,
    "min_adv_usd": 5000000,
    "min_mcap_usd": 500000000,
    "min_history_days": 252
  },
  "NASDAQ": {
    "min_price_usd": 4.50,
    "min_adv_usd": 5000000,
    "min_mcap_usd": 500000000,
    "min_history_days": 252
  },
  "XETRA": {
    "min_price_eur": 5.00,
    "min_adv_eur": 5000000,
    "min_mcap_eur": 200000000,
    "min_history_days": 252
  }
}
```

```

},
"LSE": {
    "min_price_gbp": 4.00,
    "min_adv_gbp": 5000000,
    "min_mcap_gbp": 200000000,
    "min_history_days": 252
}
}

```

Screening Logic:

```

def screen_universe(as_of_date: str) -> list:
    """
    Apply all filters to create qualified universe

    Returns: List of symbols that pass ALL filters
    """
    results = []

    for symbol in get_all_symbols():
        df = load_consolidated_data(symbol)
        company_info = load_company_info(symbol)

        # Get last 30 days of data
        lookback_df = df[df.index <= as_of_date].tail(30)

        if len(lookback_df) < 20:
            # Insufficient recent data
            continue

        # Extract exchange config
        exchange = company_info['exchange']
        config = load_filter_thresholds(exchange)

        # Filter 1: Price minimum (30-day average)
        avg_price = lookback_df['close'].mean()
        if avg_price < config['min_price']:
            continue

        # Filter 2: Average Daily Volume (30-day in local currency)
        avg_volume_local = lookback_df['volume'].mean()
        avg_dollar_volume = avg_volume_local * avg_price

        # Convert to threshold currency
        if config['currency'] == 'EUR':
            fx_rate = get_fx_rate(company_info['currency'], 'EUR', as_of_date)
            avg_volume_eur = avg_dollar_volume / fx_rate
            if avg_volume_eur < config['min_adv_eur']:
                continue
        else:
            if avg_dollar_volume < config['min_adv_usd']:
                continue

        # Filter 3: Market Cap
        market_cap = company_info['market_cap']
        if market_cap < config['min_mcap']:
            continue

        # Filter 4: Listing History
        history_days = len(df)
        if history_days < config['min_history_days']:
            continue

        # Filter 5: Data Quality
        if not df['data_quality'].iloc[-1]:
            continue

        # Filter 6: No pending corporate actions
        if has_pending_corporate_actions(symbol, as_of_date):
            continue

        # Passed all filters
        results.append({
            'symbol': symbol,
            'name': company_info['name'],
            'exchange': exchange,
            'sector': company_info['sector'],
            'market_cap': market_cap,
            'avg_price': avg_price,
            'avg_volume': avg_dollar_volume,
            'history_days': history_days,
            'screened_date': as_of_date

```

```
    })

    return results
```

2.2 TECHNICAL INDICATORS

Script 5: Indicator Calculator

File: scripts/05_calculate_indicators.py

Purpose: Calculate SMA, ATR, ADX for all qualified symbols

Inputs:

- data_cache/qualified/qualified_symbols.json
- data_cache/consolidated/*.parquet

Outputs:

- data_cache/indicators/{symbol}_indicators.parquet

Execution:

```
python scripts/05_calculate_indicators.py --as-of-date 2026-01-31
```

Indicator Formulas:

```
def calculate_sma(df: pd.DataFrame, period: int) -> pd.Series:
    """
    Simple Moving Average

    Formula:
        SMA(n) = (Close[0] + Close[1] + ... + Close[n-1]) / n

    Uses: Adjusted close prices (accounts for splits)
    """
    return df['adjusted_close'].rolling(window=period).mean()

def calculate_atr(df: pd.DataFrame, period: int = 20) -> pd.Series:
    """
    Average True Range (percentage)

    Formula:
        TR = max(High - Low, |High - Close_prev|, |Low - Close_prev|)
        ATR(n) = SMA(TR, n)
        ATR_pct = (ATR / Close) * 100

    Uses: Actual OHLC (not adjusted)
    """
    high = df['high']
    low = df['low']
    close = df['close']
    prev_close = close.shift(1)

    tr = pd.DataFrame({
        'hl': high - low,
        'hc': abs(high - prev_close),
        'lc': abs(low - prev_close)
    }).max(axis=1)

    atr = tr.rolling(window=period).mean()
    atr_pct = (atr / close) * 100

    return atr_pct

def calculate_adx(df: pd.DataFrame, period: int = 14) -> pd.Series:
    """
    Average Directional Index

    Formula (Wilder's method):
        +DM = High[0] - High[-1] if positive, else 0
        -DM = Low[-1] - Low[0] if positive, else 0

        +DI = 100 * EMA(+DM, period) / ATR(period)
        -DI = 100 * EMA(-DM, period) / ATR(period)

        DX = 100 * |+DI - -DI| / (+DI + -DI)
```

```

    ADX = EMA(DX, period)

Returns: ADX value (0-100 scale, >20 = trending)
"""
high = df['high']
low = df['low']
close = df['close']

# Directional movement
plus_dm = high.diff()
minus_dm = -low.diff()

plus_dm[plus_dm < 0] = 0
minus_dm[minus_dm < 0] = 0

# True range
prev_close = close.shift(1)
tr = pd.DataFrame({
    'hl': high - low,
    'hc': abs(high - prev_close),
    'lc': abs(low - prev_close)
}).max(axis=1)

# Smoothed averages (Wilder's EMA with alpha = 1/period)
atr = tr.ewm(alpha=1/period, adjust=False).mean()
plus_di = 100 * plus_dm.ewm(alpha=1/period, adjust=False).mean() / atr
minus_di = 100 * minus_dm.ewm(alpha=1/period, adjust=False).mean() / atr

# Directional index
dx = 100 * abs(plus_di - minus_di) / (plus_di + minus_di)
adx = dx.ewm(alpha=1/period, adjust=False).mean()

return adx

def calculate_all_indicators(symbol: str, as_of_date: str) -> pd.DataFrame:
    """
    Calculate all required indicators for a symbol
    """
    df = load_consolidated_data(symbol)

    # Ensure we have enough history (300 days for 200 SMA + buffer)
    min_required_days = 300
    if len(df) < min_required_days:
        raise ValueError(f"{symbol}: Insufficient history ({len(df)} < {min_required_days})")

    # Calculate indicators
    df['sma_50'] = calculate_sma(df, 50)
    df['sma_200'] = calculate_sma(df, 200)
    df['atr_20_pct'] = calculate_atr(df, 20)
    df['adx_14'] = calculate_adx(df, 14)

    # Only return data up to as_of_date
    df = df[df.index <= as_of_date]

    # Save indicators
    output_path = f"data_cache/indicators/{symbol}_indicators.parquet"
    df.to_parquet(output_path, compression='snappy')

    return df

```

2.3 TREND QUALIFICATION

Script 6: Trend Qualifier

File: scripts/06_qualify_trends.py

Purpose: Identify instruments in confirmed uptrends

Inputs:

- data_cache/indicators/*.parquet

Outputs:

- data_cache/signals/qualified_trends.json

Execution:

```
python scripts/06_qualify_trends.py --as-of-date 2026-01-31
```

Trend Qualification Rules (ALL must be TRUE):

```
def is_trend_qualified(df: pd.DataFrame, as_of_date: str) -> tuple:
    """
    Check if instrument is in qualified uptrend

    Rules (ALL must pass):
    1. SMA_50 > SMA_200 (golden cross)
    2. Close > SMA_50 (price above short-term trend)
    3. ADX_14 > 20 (sufficient trend strength)

    Returns: (qualified: bool, reason: str)
    """
    # Get most recent data point as of date
    latest = df[df.index <= as_of_date].iloc[-1]

    # Rule 1: Golden cross
    if latest['sma_50'] <= latest['sma_200']:
        return False, "SMA_50 not above SMA_200"

    # Rule 2: Price above SMA_50
    if latest['close'] <= latest['sma_50']:
        return False, "Close not above SMA_50"

    # Rule 3: Trend strength
    if latest['adx_14'] <= 20:
        return False, f"ADX too low ({latest['adx_14']:.1f})"

    return True, "Qualified"

def qualify_all_trends(as_of_date: str) -> dict:
    """
    Run trend qualification for all symbols
    """
    qualified_symbols = load_qualified_symbols()
    results = {}

    for symbol in qualified_symbols:
        try:
            df = load_indicators(symbol)
            qualified, reason = is_trend_qualified(df, as_of_date)

            if qualified:
                latest = df[df.index <= as_of_date].iloc[-1]
                results[symbol] = {
                    'symbol': symbol,
                    'qualified': True,
                    'sma_50': float(latest['sma_50']),
                    'sma_200': float(latest['sma_200']),
                    'close': float(latest['close']),
                    'adx_14': float(latest['adx_14']),
                    'atr_20_pct': float(latest['atr_20_pct']),
                    'qualified_date': as_of_date
                }
        except Exception as e:
            logger.error(f"Failed to qualify {symbol}: {e}")
            continue

    return results
```

2.4 MOMENTUM RANKING

Script 7: Momentum Ranker

File: scripts/07_rank_momentum.py

Purpose: Rank qualified trends by momentum score

Inputs:

- data_cache/signals/qualified_trends.json
- data_cache/indicators/*.parquet

Outputs:

- data_cache/signals/momentum_ranked.json

Execution:

```
python scripts/07_rank_momentum.py --as-of-date 2026-01-31
```

Momentum Score Formula (EXPLICIT - FIXED FROM RECOMMENDATIONS):

```
def calculate_momentum_score(df: pd.DataFrame, as_of_date: str) -> float:
    """
    Calculate momentum score for ranking

    Formula:
    Momentum Score = (Close - SMA_200) / SMA_200 × 100

    Interpretation:
    - Positive score = uptrend (above 200-day SMA)
    - Higher score = stronger momentum
    - Example: Score of 15.0 = price is 15% above 200-day SMA

    Alternative formulas considered but rejected:
    - Rate of Change (3m, 6m, 12m): Too sensitive to entry timing
    - Relative Strength Index: Not comparative across instruments
    - Price / SMA_50: Too short-term, whipsaw-prone

    Rationale for chosen formula:
    - Simple and transparent
    - Captures long-term trend strength
    - Comparable across all instruments
    - Academic support (Moskowitz et al., 2012)
    """
    latest = df[df.index <= as_of_date].iloc[-1]

    momentum_score = ((latest['close'] - latest['sma_200']) / latest['sma_200']) * 100

    return float(momentum_score)


def rank_by_momentum(as_of_date: str) -> list:
    """
    Rank all qualified trends by momentum score

    Returns: List of dicts sorted by momentum (descending)
    """
    qualified_trends = load_qualified_trends(as_of_date)
    results = []

    for symbol, trend_data in qualified_trends.items():
        df = load_indicators(symbol)
        momentum_score = calculate_momentum_score(df, as_of_date)

        results.append({
            'symbol': symbol,
            'momentum_score': momentum_score,
            'sma_50': trend_data['sma_50'],
            'sma_200': trend_data['sma_200'],
            'close': trend_data['close'],
            'adx_14': trend_data['adx_14'],
            'atr_20_pct': trend_data['atr_20_pct']
        })

    # Sort by momentum score (descending)
    results.sort(key=lambda x: x['momentum_score'], reverse=True)

    return results
```

2.5 ENTRY SIGNAL (EXPLICIT - FIXED FROM RECOMMENDATIONS)

CRITICAL FIX: Entry timing was underspecified in v2.0/v3.0. Now explicitly defined:

```
"""
ENTRY SIGNAL RULES (No Discretion)

Entry occurs when ALL of the following are TRUE:

1. TIMING:
    - Today is the rebalancing date (last trading day of month)

2. QUALIFICATION:
    - Instrument passed trend qualification as of rebalancing date:
      * SMA_50 > SMA_200
      * Close > SMA_50
```

```

* ADX_14 > 20

3. MOMENTUM RANKING:
- Instrument is in top N by momentum score
- N = get_max_positions(account_equity)

4. PORTFOLIO CONSTRAINT:
- Instrument is not already in portfolio OR
- Instrument is in portfolio but needs rebalancing (size adjustment)

EXECUTION:
- Entry order: Limit order at Close × 1.005 (close + 0.5%)
- Order placed: End of rebalancing day
- Order executed: Next trading day (first trading day of new month)
- If limit not filled in 2 days: Cancel order, skip position

EXAMPLE:
Rebalancing Date: Jan 31, 2026 (Friday)
Signal Generated: Jan 31, 2026 at market close
Entry Price: Jan 31 close × 1.005
Order Placed: Jan 31, 2026 after close
Expected Execution: Feb 3, 2026 (Monday) if price ≤ limit

NO ENTRIES BETWEEN REBALANCING DATES (monthly only)
"""

def generate_entry_signals(
    momentum_ranked: list,
    current_positions: dict,
    account_equity: float,
    rebalancing_date: str
) -> list:
    """
    Generate entry signals for monthly rebalancing

    Returns: List of symbols to enter with entry prices
    """
    max_positions = get_max_positions(account_equity)

    # Top N by momentum
    top_n = momentum_ranked[:max_positions]
    top_n_symbols = [s['symbol'] for s in top_n]

    # Identify new entries (not in current portfolio)
    new_entries = []
    for symbol_data in top_n:
        symbol = symbol_data['symbol']

        if symbol not in current_positions:
            # New position
            close_price = symbol_data['close']
            limit_price = close_price * 1.005 # +0.5%

            new_entries.append({
                'symbol': symbol,
                'action': 'ENTRY',
                'close_price': close_price,
                'limit_price': limit_price,
                'order_type': 'LIMIT',
                'momentum_score': symbol_data['momentum_score'],
                'signal_date': rebalancing_date,
                'execution_window': '2_trading_days'
            })

    return new_entries

```

2.6 POSITION SIZING (ENHANCED - FIXED FROM RECOMMENDATIONS)

Script 8: Position Sizer

File: scripts/08_calculate_position_sizes.py

Purpose: Calculate volatility-adjusted position sizes

Inputs:

- data_cache/signals/momentum_ranked.json
- Portfolio equity (user input or from portfolio state)

Outputs:

- data_cache/portfolio/position_sizes.json

Execution:

```
python scripts/08_calculate_position_sizes.py \  
--account-equity 50000 \  
--as-of-date 2026-01-31 \  
--max-positions 25
```

Position Sizing Formula (EXPLICIT - ENHANCED):

```
def calculate_position_size(  
    account_equity: float,  
    instrument_atr_pct: float,  
    median_atr_pct: float,  
    instrument_price: float  
) -> dict:  
    """  
    Calculate volatility-adjusted position size  
  
    Formula:  
    Step 1: Base Risk  
        Base_Risk = Account_Equity × Target_Risk_Per_Position  
        Target_Risk_Per_Position = 2.0% (constant)  
  
    Step 2: Volatility Adjustment  
        Volatility_Multiplier = Median_ATR / Instrument_ATR  
  
    Rationale:  
        - Lower volatility → larger position (less risky)  
        - Higher volatility → smaller position (more risky)  
        - Normalized across all positions  
  
    Step 3: Calculate Raw Position Value  
        Raw_Position_Value = Base_Risk × Volatility_Multiplier  
  
    Step 4: Apply Floor and Ceiling  
        Final_Position_Value = CLIP(  
            Raw_Position_Value,  
            min = 0.5% × Account_Equity,  
            max = 8.0% × Account_Equity  
        )  
  
    Step 5: Convert to Shares  
        Shares = floor(Final_Position_Value / Instrument_Price)  
        Actual_Position_Value = Shares × Instrument_Price  
  
    Rationale:  
        - Target 2% risk per position (industry standard for trend following)  
        - Inverse ATR weighting (lower volatility = larger size)  
        - Floor (0.5%) prevents dust positions  
        - Ceiling (8.0%) prevents concentration risk  
        - Floor operation prevents fractional shares  
  
    Example:  
        Account = €50,000  
        Instrument ATR = 2.5%  
        Median ATR = 2.0%  
        Price = €100  
  
        Step 1: Base_Risk = 50,000 × 0.02 = €1,000  
        Step 2: Vol_Multiplier = 2.0 / 2.5 = 0.8  
        Step 3: Raw_Value = 1,000 × 0.8 = €800  
        Step 4: Min = 50,000 × 0.005 = €250  
                 Max = 50,000 × 0.08 = €4,000  
                 Final_Value = €800 (within bounds)  
        Step 5: Shares = floor(800 / 100) = 8  
                 Actual_Value = 8 × 100 = €800 (1.6% of account)  
    """  
    TARGET_RISK = 0.02 # 2% per position  
    MIN_POSITION_PCT = 0.005 # 0.5% minimum  
    MAX_POSITION_PCT = 0.08 # 8.0% maximum  
  
    # Step 1: Base risk  
    base_risk = account_equity * TARGET_RISK  
  
    # Step 2: Volatility adjustment  
    volatility_multiplier = median_atr_pct / instrument_atr_pct  
    raw_position_value = base_risk * volatility_multiplier  
  
    # Step 3: Apply floor and ceiling  
    min_value = account_equity * MIN_POSITION_PCT
```

```

max_value = account_equity * MAX_POSITION_PCT
final_position_value = max(min_value, min(raw_position_value, max_value))

# Step 4: Calculate shares
shares = int(final_position_value / instrument_price)
actual_position_value = shares * instrument_price

# Step 5: Calculate position percentage
position_pct = (actual_position_value / account_equity) * 100

return {
    'shares': shares,
    'position_value_eur': actual_position_value,
    'position_pct': position_pct,
    'base_risk': base_risk,
    'volatility_multiplier': volatility_multiplier,
    'raw_value': raw_position_value,
    'min_value': min_value,
    'max_value': max_value,
    'clipped': (raw_position_value != final_position_value)
}

def size_portfolio(
    ranked_symbols: list,
    account_equity: float,
    max_positions: int
) -> dict:
    """
    Calculate position sizes for top N symbols

    Process:
    1. Select top N by momentum score
    2. Calculate median ATR across selected symbols
    3. Size each position using volatility adjustment
    4. Validate portfolio constraints
    """
    # Step 1: Select top N
    selected = ranked_symbols[:max_positions]

    # Step 2: Calculate median ATR
    atr_values = [s['atr_20_pct'] for s in selected]
    median_atr = np.median(atr_values)

    # Step 3: Size each position
    results = {}
    for symbol_data in selected:
        size = calculate_position_size(
            account_equity=account_equity,
            instrument_atr_pct=symbol_data['atr_20_pct'],
            median_atr_pct=median_atr,
            instrument_price=symbol_data['close']
        )

        results[symbol_data['symbol']] = {
            **symbol_data,
            **size,
            'entry_price': symbol_data['close']
        }

    # Step 4: Validate constraints
    total_allocation = sum(r['position_value_eur'] for r in results.values())
    allocation_pct = (total_allocation / account_equity) * 100

    if allocation_pct > 95:
        logger.warning(f"High allocation: {allocation_pct:.1f}%")

    return results

```

2.7 STOP-LOSS RULES (CLARIFIED - FIXED FROM RECOMMENDATIONS)

Script 9: Stop-Loss Calculator

File: scripts/09_calculate_stops.py

Purpose: Calculate initial and trailing stop-loss levels

Inputs:

- data_cache/portfolio/position_sizes.json

Outputs:

- data_cache/portfolio/stop_levels.json

Execution:

```
python scripts/09_calculate_stops.py --as-of-date 2026-01-31
```

Stop-Loss Formulas (EXPLICIT - CLARIFIED):

```
def calculate_initial_stop(entry_price: float, atr: float) -> dict:
    """
    Calculate initial stop-loss at entry

    Formula:
        Initial_Stop = Entry_Price - (3.0 × ATR)

    Characteristics:
        - Set at entry, never moves down
        - Fixed 3.0× ATR multiple (tight)
        - Typical stop distance: 6–9% below entry for stocks
        - Exit triggered if close ≤ stop_price on ANY day

    Example:
        Entry = €100
        ATR = €2.50
        Initial_Stop = 100 - (3.0 × 2.50) = €92.50
        Stop Distance = 7.5%
    """
    INITIAL_STOP_MULTIPLIER = 3.0

    stop_price = entry_price - (INITIAL_STOP_MULTIPLIER * atr)
    stop_distance_pct = ((entry_price - stop_price) / entry_price) * 100

    return {
        'stop_price': stop_price,
        'stop_distance_pct': stop_distance_pct,
        'atr_multiple': INITIAL_STOP_MULTIPLIER,
        'type': 'initial',
        'never_moves_down': True,
        'update_frequency': 'set_once_at_entry'
    }

def calculate_trailing_stop(
    current_price: float,
    entry_price: float,
    current_atr: float,
    current_trailing_stop: float = None
) -> dict:
    """
    Calculate trailing stop-loss (activates after profit threshold)

    Activation Rule:
        Trailing stop activates when: Current_Price ≥ 1.15 × Entry_Price
        (i.e., +15% profit from entry)

    Update Frequency:
        Every Friday market close (weekly)

    Calculation Formula:
        New_Trailing_Stop = max(
            Current_Trailing_Stop,
            Friday_Close - (4.0 × Friday_ATR)
        )

    Exit Rule:
        IF (Daily_Close < Trailing_Stop) THEN
            Exit at next day's market open (market order)

    Characteristics:
        - Only moves UP, never down (ratchet mechanism)
        - Updated weekly (Friday close only)
        - Wider than initial stop (4.0× vs 3.0× ATR)
        - Gives trend room to breathe while locking in profits

    Example:
        Entry = €100
        Current_Price = €120 (+20% profit, trailing activated)
        Current_ATR = €3.00
        Current_Trailing_Stop = €110 (from last week)
```

```

    New_Trailing_Stop = max(110, 120 - 4.0 × 3.00)
                        = max(110, 108)
                        = €110 (doesn't move down!)

    Next week, if price = €125, ATR = €3.20:
    New_Trailing_Stop = max(110, 125 - 4.0 × 3.20)
                        = max(110, 112.20)
                        = €112.20 (moves up to lock in gains)
"""
ACTIVATION_THRESHOLD = 1.15 # 15% profit
TRAILING_STOP_MULTIPLIER = 4.0

# Check if trailing stop should be activated
profit_pct = ((current_price - entry_price) / entry_price)
trailing_active = (profit_pct >= (ACTIVATION_THRESHOLD - 1.0))

if not trailing_active:
    return {
        'active': False,
        'reason': f'Profit {profit_pct*100:.1f}% < 15% threshold',
        'threshold_price': entry_price * ACTIVATION_THRESHOLD
    }

# Calculate new trailing stop
calculated_stop = current_price - (TRAILING_STOP_MULTIPLIER * current_atr)

# Trailing stop only moves UP (ratchet mechanism)
if current_trailing_stop is None:
    # First time activating trailing stop
    new_stop = calculated_stop
else:
    # Use maximum of current stop and calculated stop
    new_stop = max(current_trailing_stop, calculated_stop)

stop_distance_pct = ((current_price - new_stop) / current_price) * 100

return {
    'active': True,
    'stop_price': new_stop,
    'stop_distance_pct': stop_distance_pct,
    'atr_multiple': TRAILING_STOP_MULTIPLIER,
    'type': 'trailing',
    'only_moves_up': True,
    'update_frequency': 'weekly_friday_close',
    'profit_since_entry_pct': profit_pct * 100,
    'moved_up': (current_trailing_stop is not None and new_stop > current_trailing_stop)
}

def calculate_all_stops(positions: dict, as_of_date: str) -> dict:
    """
    Calculate stops for all positions

    Process:
    1. For new positions: set initial stop (one-time)
    2. For existing positions: check if trailing should activate
    3. If trailing active: update stop level (only on Fridays)
    4. Return stop prices for all positions
    """
    results = {}

    # Check if today is Friday (for trailing stop updates)
    is_friday = pd.to_datetime(as_of_date).dayofweek == 4

    for symbol, position in positions.items():
        df = load_indicators(symbol)
        latest = df[df.index <= as_of_date].iloc[-1]

        # Calculate ATR in absolute terms (not percentage)
        atr_abs = (latest['atr_20_pct'] / 100) * latest['close']

        # Determine if this is a new position or existing
        if position.get('is_new_entry', True):
            # New position: set initial stop (one-time)
            stop = calculate_initial_stop(
                entry_price=position['entry_price'],
                atr=atr_abs
            )
            stop['last_update_date'] = as_of_date
        else:
            # Existing position: check trailing
            current_stop = position.get('current_stop_price')

```

```

trailing_info = calculate_trailing_stop(
    current_price=latest['close'],
    entry_price=position['entry_price'],
    current_atr=atr_abs,
    current_trailing_stop=position.get('trailing_stop_price')
)

if trailing_info.get('active', False):
    # Trailing stop is active
    if is_friday:
        # Update on Fridays only
        stop = trailing_info
        stop['last_update_date'] = as_of_date
    else:
        # Not Friday, keep current stop
        stop = {
            'stop_price': position.get('trailing_stop_price', current_stop),
            'type': 'trailing',
            'active': True,
            'last_update_date': position.get('stop_last_update_date'),
            'next_update': 'next_friday'
        }
    else:
        # Trailing not active, use initial stop
        stop = {
            'stop_price': position.get('initial_stop_price'),
            'type': 'initial',
            'active': True,
            'last_update_date': position.get('stop_last_update_date'),
            'trailing_activation_price': trailing_info.get('threshold_price')
        }

results[symbol] = stop

return results

```

2.8 EXIT RULES

Script 10: Exit Signal Generator

File: scripts/10_generate_exit_signals.py

Purpose: Identify positions that should be exited

Inputs:

- Current portfolio positions (from portfolio state)
- data_cache/indicators/*.parquet
- data_cache/portfolio/stop_levels.json

Outputs:

- data_cache/signals/exit_signals.json

Execution:

```
python scripts/10_generate_exit_signals.py --as-of-date 2026-01-31
```

Exit Rules (Prioritized):

```

def check_exit_conditions(
    symbol: str,
    position: dict,
    df: pd.DataFrame,
    as_of_date: str
) -> dict:
    """
    Check all exit conditions for a position

    Exit Priority (checked in order):
    1. Stop-loss hit (highest priority)
    2. Trend reversal (SMA death cross)
    3. Trend weakness (ADX collapse)
    4. Rebalancing rotation (dropped from top N)

    Returns: (exit: bool, reason: str, priority: int)
    """
    latest = df[df.index <= as_of_date].iloc[-1]

```

```

# EXIT 1: Stop-loss hit
stop_price = position['current_stop_price']
if latest['close'] <= stop_price:
    return {
        'exit': True,
        'reason': 'stop_loss_hit',
        'priority': 1,
        'stop_price': stop_price,
        'close_price': latest['close'],
        'exit_type': 'market_order_next_open',
        'detail': f"Close {latest['close']:.2f} <= Stop {stop_price:.2f}"
    }

# EXIT 2: Trend reversal (death cross)
if latest['sma_50'] < latest['sma_200']:
    return {
        'exit': True,
        'reason': 'trend_reversal',
        'priority': 2,
        'sma_50': latest['sma_50'],
        'sma_200': latest['sma_200'],
        'exit_type': 'market_order_next_open',
        'detail': f"SMA50 {latest['sma_50']:.2f} < SMA200 {latest['sma_200']:.2f}"
    }

# EXIT 3: Trend weakness (ADX collapse)
# Check if ADX < 15 for 3 consecutive days
recent_adx = df[df.index <= as_of_date]['adx_14'].tail(3)
if len(recent_adx) >= 3 and (recent_adx < 15).all():
    return {
        'exit': True,
        'reason': 'trend_weakness',
        'priority': 3,
        'adx_values': recent_adx.tolist(),
        'exit_type': 'market_order_next_open',
        'detail': f"ADX < 15 for 3 consecutive days: {recent_adx.tolist()}"
    }

# EXIT 4: Rebalancing rotation
# (This is checked separately in rebalancing script)
# If position is not in top N by momentum, exit at month-end

# No exit triggered
return {
    'exit': False,
    'reason': 'holding',
    'priority': None
}

def generate_all_exit_signals(
    current_positions: dict,
    as_of_date: str
) -> dict:
    """
    Check exit conditions for all current positions
    """
    exit_signals = {}

    for symbol, position in current_positions.items():
        df = load_indicators(symbol)
        exit_check = check_exit_conditions(symbol, position, df, as_of_date)

        if exit_check['exit']:
            exit_signals[symbol] = exit_check

    return exit_signals

```

2.9 REBALANCING TIMING (EXPLICIT - FIXED FROM RECOMMENDATIONS)

CRITICAL FIX: Rebalancing timing was ambiguous in v2.0/v3.0. Now explicitly defined:

```

"""
REBALANCING SCHEDULE (No Discretion)

Frequency: Monthly

Anchor Date: Last trading day of each calendar month

Cutoff Time: Market close of last trading day

```

Holiday Handling:

IF last calendar day of month is non-trading day (weekend/holiday)
THEN use last trading day before month-end

Example:

- January 31, 2026 is Saturday
- Last trading day is January 29, 2026 (Friday)
- Rebalancing signals generated using January 29 close prices

Execution Window:

- Signals generated: Rebalancing date at market close
- Exit orders: First trading day of new month (market orders at open)
- Entry orders: First trading day of new month (limit orders, close + 0.5%)
- Stop-loss orders: Set immediately after entry fills

Order Priority:

1. Execute all exits first (free up capital)
2. Execute entries with freed capital
3. Set stop-losses for all new positions
4. Update trailing stops for existing positions (if Friday)

Example Timeline:

- Rebalancing Date: January 31, 2026 (Friday)
- Signal Generation: January 31, 2026 at 4:00 PM EST
- Order Preparation: January 31, 2026 evening (manual review)
- Order Execution: February 3, 2026 (Monday) at 9:30 AM EST

"""

```
REBALANCING_RULES = {
    'frequency': 'monthly',
    'anchor': 'last_trading_day_of_month',
    'cutoff_time': 'market_close',
    'execution_day': 'first_trading_day_of_next_month',

    'holiday_handling': {
        'rule': 'use_last_trading_day_before_month_end',
        'example': 'If Jan 31 is Saturday, use Jan 29 (Friday) close'
    },

    'execution_timing': {
        'exits': 'next_day_open_market_order',
        'entries': 'next_day_limit_order_close_plus_0.5_pct',
        'stops': 'set_immediately_after_fills'
    },

    'order_priority': [
        '1. Execute all exits first (free up capital)',
        '2. Execute entries with freed capital',
        '3. Update stop-losses for all positions'
    ]
}
```

```
def get_rebalancing_date(year: int, month: int) -> datetime:
    """
    Calculate rebalancing date for a given month

    Returns: Last trading day of the month
    """
    # Get last calendar day of month
    last_day = calendar.monthrange(year, month)[1]
    candidate_date = datetime(year, month, last_day)

    # Find last trading day (walk backwards if weekend/holiday)
    while not is_trading_day(candidate_date):
        candidate_date -= timedelta(days=1)

    return candidate_date

def get_execution_date(rebalancing_date: datetime) -> datetime:
    """
    Calculate execution date (first trading day of next month)
    """
    next_month = rebalancing_date + timedelta(days=1)

    # Find first trading day
    while not is_trading_day(next_month):
        next_month += timedelta(days=1)

    return next_month
```

2.10 CIRCUIT BREAKERS (NEW - FROM RECOMMENDATIONS)

CRITICAL ADDITION: Prevent catastrophic losses during extreme market conditions

```
"""
CIRCUIT BREAKERS (Automatic Halt Conditions)

Purpose: Prevent system from executing into disaster during extreme conditions

When Triggered: Halt all NEW entries, allow only risk-reducing exits

Conditions:

1. PORTFOLIO DRAWDOWN:
  IF (Current_Equity / Peak_Equity - 1) < -15%
  THEN halt entries for 5 trading days

  Rationale: Large drawdown indicates strategy not working in current regime
  Action: Stop adding risk, let existing positions work out

2. VIX SPIKE (if trading US stocks):
  IF VIX > 40
  THEN halt entries until VIX < 30 for 3 consecutive days

  Rationale: Extreme volatility = unreliable signals
  Action: Wait for market to stabilize

3. CORRELATION BREAKDOWN:
  IF pairwise correlation of top 10 positions > 0.85
  THEN halt entries, flag for review

  Rationale: High correlation = concentrated risk, not diversified
  Action: Review position selection, pause new entries

4. CONCENTRATION CREEP:
  IF top 3 positions > 30% of portfolio
  THEN halt entries, force rebalancing

  Rationale: Excessive concentration violates risk limits
  Action: Exit excess positions before adding new ones

5. DATA STALENESS:
  IF any position has data > 3 trading days old
  THEN halt all trading, alert operator

  Rationale: Stale data = trading blind
  Action: Fix data feed before resuming

Manual Override:
- Human can override circuit breakers after review
- Override must be logged with rationale
- Override expires after 1 trading day

Implementation:
- Check circuit breakers BEFORE generating rebalancing recommendations
- Log all circuit breaker triggers
- Send alerts to operator
- Resume automatically when conditions clear
"""

class CircuitBreaker:
    """
    Monitor portfolio health and halt trading when needed
    """

    def check_all_breakers(
        self,
        current_equity: float,
        peak_equity: float,
        current_positions: dict,
        vix_level: float = None
    ) -> dict:
        """
        Check all circuit breaker conditions

        Returns: (halted: bool, reasons: list, actions: list)
        """
        breakers_triggered = []

        # Breaker 1: Portfolio drawdown
        drawdown = (current_equity / peak_equity - 1)
```

```

if drawdown < -0.15:
    breakers_triggered.append({
        'breaker': 'portfolio_drawdown',
        'severity': 'HIGH',
        'value': drawdown,
        'threshold': -0.15,
        'action': 'halt_entries_5_days',
        'detail': f"Drawdown {drawdown:.1%} exceeds -15% threshold"
    })

# Breaker 2: VIX spike (if applicable)
if vix_level is not None and vix_level > 40:
    breakers_triggered.append({
        'breaker': 'vix_spike',
        'severity': 'MEDIUM',
        'value': vix_level,
        'threshold': 40,
        'action': 'halt_entries_until_vix_below_30',
        'detail': f"VIX {vix_level:.1f} exceeds 40 threshold"
    })

# Breaker 3: Correlation breakdown
if len(current_positions) >= 10:
    position_values = [p for p in current_positions.values()]
    top_10 = sorted(position_values, key=lambda x: x['current_value'], reverse=True)[:10]

    # Calculate pairwise correlations (simplified)
    # In real implementation, use returns correlation
    correlation_matrix = calculate_correlation_matrix(top_10)
    max_correlation = correlation_matrix.max()

    if max_correlation > 0.85:
        breakers_triggered.append({
            'breaker': 'correlation_breakdown',
            'severity': 'MEDIUM',
            'value': max_correlation,
            'threshold': 0.85,
            'action': 'halt_entries_review_required',
            'detail': f"Max correlation {max_correlation:.2f} exceeds 0.85"
        })

# Breaker 4: Concentration creep
position_values = [p['current_value'] for p in current_positions.values()]
position_values.sort(reverse=True)
top_3_pct = sum(position_values[:3]) / current_equity

if top_3_pct > 0.30:
    breakers_triggered.append({
        'breaker': 'concentration_creep',
        'severity': 'HIGH',
        'value': top_3_pct,
        'threshold': 0.30,
        'action': 'halt_entries_force_rebalancing',
        'detail': f"Top 3 positions {top_3_pct:.1%} exceed 30% threshold"
    })

# Breaker 5: Data staleness
for symbol, position in current_positions.items():
    last_update = position.get('data_last_update')
    if last_update:
        days_stale = (datetime.now().date() - last_update.date()).days
        if days_stale > 3:
            breakers_triggered.append({
                'breaker': 'data_staleness',
                'severity': 'CRITICAL',
                'symbol': symbol,
                'value': days_stale,
                'threshold': 3,
                'action': 'halt_all_trading',
                'detail': f"{symbol} data {days_stale} days old"
            })

return {
    'halted': len(breakers_triggered) > 0,
    'breakers': breakers_triggered,
    'timestamp': datetime.now().isoformat()
}

```

3.1 PORTFOLIO CONSTRAINTS

```
PORTFOLIO_CONSTRAINTS = {
    'position_limits': {
        'max_per_position_pct': 8.0,
        'min_per_position_pct': 0.5,
        'target_per_position_pct': 4.0 # typical with 25 positions
    },

    'asset_class_limits': {
        'etf': {
            'min_pct': 0,
            'target_pct': 38,
            'max_pct': 45
        },
        'stock': {
            'min_pct': 0,
            'target_pct': 42,
            'max_pct': 50
        },
        'crypto': {
            'min_pct': 0,
            'target_pct': 15,
            'max_pct': 20
        }
    },

    'liquidity_limits': {
        'max_position_as_pct_of_adv': 5.0,
        'min_cash_reserve_pct': 5.0
    },

    'concentration_limits': {
        'max_single_sector_pct': 30.0,
        'warning_correlation_threshold': 0.7 # warn if top 3 corr > 0.7
    },

    'position_count': {
        'account_10k': 10,
        'account_25k': 15,
        'account_50k': 20,
        'account_100k': 25
    }
}
```

```
def get_max_positions(account_equity: float) -> int:
    """
    Determine max positions based on account size

    Rationale:
    - Small accounts: fewer positions (reduce transaction costs)
    - Large accounts: more positions (diversification)
    """
    if account_equity < 25000:
        return 10
    elif account_equity < 50000:
        return 15
    elif account_equity < 100000:
        return 20
    else:
        return 25
```

3.2 MONTHLY REBALANCING SCRIPT

Script 11: Monthly Rebalancer

File: scripts/11_monthly_rebalancing.py

Purpose: Generate monthly rebalancing recommendations

Inputs:

- Current portfolio positions (data/portfolio_state.json)
- data_cache/signals/momentum_ranked.json
- data_cache/portfolio/position_sizes.json
- data_cache/signals/exit_signals.json
- Account equity (user input)

Outputs:

- reports/rebalancing/{YYYY-MM}_recommendations.json
- reports/rebalancing/{YYYY-MM}_recommendations.pdf
- reports/rebalancing/{YYYY-MM}_recommendations.csv

Execution:

```
python scripts/11_monthly_rebalancing.py \  
--account-equity 50000 \  
--rebalance-date 2026-01-31
```

Rebalancing Logic:

```
def generate_rebalancing_recommendations(  
    account_equity: float,  
    rebalance_date: str  
) -> dict:  
    """  
    Generate monthly rebalancing recommendations  
  
    Process:  
    1. Check circuit breakers  
    2. Load current portfolio positions  
    3. Check exit signals (stop-loss, trend reversal, etc.)  
    4. Load momentum-ranked universe  
    5. Select top N new positions  
    6. Calculate position sizes for new entries  
    7. Identify positions to exit (not in top N anymore)  
    8. Generate recommendation report  
    9. Await human approval  
    """  
    # 1. Check circuit breakers FIRST  
    breaker = CircuitBreaker()  
    breaker_status = breaker.check_all_breakers(  
        current_equity=account_equity,  
        peak_equity=get_peak_equity(),  
        current_positions=load_portfolio_state(),  
        vix_level=get_current_vix()  
    )  
  
    if breaker_status['halted']:  
        return {  
            'status': 'HALTED',  
            'circuit_breakers': breaker_status['breakers'],  
            'message': 'Trading halted due to circuit breaker trigger',  
            'action_required': 'Review conditions before resuming'  
        }  
  
    # 2. Load current state  
    current_positions = load_portfolio_state()  
  
    # 3. Check mandatory exits  
    exit_signals = load_exit_signals(rebalance_date)  
    mandatory_exits = [  
        symbol for symbol, signal in exit_signals.items()  
        if signal['priority'] in [1, 2, 3] # Stop-loss, reversal, weakness  
    ]  
  
    # 4. Load momentum-ranked universe  
    ranked_universe = load_momentum_ranked(rebalance_date)  
  
    # 5. Determine max positions for account size  
    max_positions = get_max_positions(account_equity)  
  
    # 6. Select top N  
    top_n = ranked_universe[:max_positions]  
    top_n_symbols = [s['symbol'] for s in top_n]  
  
    # 7. Identify rotation exits (in current portfolio but not in top N)  
    rotation_exits = [  
        symbol for symbol in current_positions.keys()  
        if symbol not in top_n_symbols and symbol not in mandatory_exits  
    ]  
  
    # 8. Identify new entries (in top N but not in current portfolio)  
    new_entries = [  
        symbol for symbol in top_n_symbols  
        if symbol not in current_positions  
    ]
```

```

# 9. Size new positions
new_position_sizes = size_portfolio(
    ranked_symbols=[s for s in top_n if s['symbol'] in new_entries],
    account_equity=account_equity,
    max_positions=len(new_entries)
)

# 10. Calculate stops for new positions
new_stops = calculate_all_stops(new_position_sizes, rebalance_date)

# 11. Compile recommendations
recommendations = {
    'rebalance_date': rebalance_date,
    'account_equity': account_equity,
    'max_positions': max_positions,
    'circuit_breakers': breaker_status,

    'exits': {
        'mandatory': [
            {
                'symbol': symbol,
                'reason': exit_signals[symbol]['reason'],
                'priority': exit_signals[symbol]['priority'],
                'detail': exit_signals[symbol]['detail'],
                'order_type': 'market_order_at_open'
            }
            for symbol in mandatory_exits
        ],
        'rotation': [
            {
                'symbol': symbol,
                'reason': 'dropped_from_top_n',
                'priority': 4,
                'detail': f"Not in top {max_positions} by momentum",
                'order_type': 'market_order_at_close'
            }
            for symbol in rotation_exits
        ]
    },

    'entries': [
        {
            'symbol': symbol,
            'momentum_score': sizes['momentum_score'],
            'rank': idx + 1,
            'shares': sizes['shares'],
            'entry_price': sizes['entry_price'],
            'position_value': sizes['position_value_eur'],
            'position_pct': sizes['position_pct'],
            'initial_stop': new_stops[symbol]['stop_price'],
            'order_type': 'limit_order_close_plus_0.5_pct',
            'rationale': {
                'sma_50': sizes['sma_50'],
                'sma_200': sizes['sma_200'],
                'adx_14': sizes['adx_14'],
                'atr_20_pct': sizes['atr_20_pct']
            }
        }
        for idx, (symbol, sizes) in enumerate(new_position_sizes.items())
    ],

    'holds': [
        {
            'symbol': symbol,
            'action': 'hold',
            'current_value': position['current_value'],
            'unrealized_pnl': position['unrealized_pnl'],
            'current_stop': position['current_stop_price']
        }
        for symbol, position in current_positions.items()
        if symbol in top_n_symbols and symbol not in mandatory_exits
    ]
}

return recommendations

```

PART 4: VISUALIZATION & MONITORING

4.1 INTERACTIVE CHART GENERATOR (NEW SCRIPT)

Script 15: Technical Analysis Charts

File: scripts/15_generate_technical_charts.py

Purpose: Generate interactive charts with price, SMA, volume, ADX, ATR for selected tickers

Inputs:

- data_cache/indicators/*.parquet
- User-selected tickers (command line or interactive)

Outputs:

- reports/charts/technical_analysis_{timestamp}.html (single-file dashboard)
- Interactive Plotly charts with 200-day lookback

Features:

- **Performant:** Loads data once, caches in memory
- **Interactive:** Click to switch between tickers
- **Comprehensive:** Price, SMA50, SMA200, Volume, ADX, ATR in single view
- **Flexible:** Select specific tickers or use default top 20

Execution:

```
# Generate charts for top 20 by momentum
python scripts/15_generate_technical_charts.py

# Generate charts for specific tickers
python scripts/15_generate_technical_charts.py --symbols AAPL.US,MSFT.US,GOOGL.US

# Generate charts for all qualified trends
python scripts/15_generate_technical_charts.py --filter all

# Custom lookback period
python scripts/15_generate_technical_charts.py --lookback 300

# Interactive mode (select from list)
python scripts/15_generate_technical_charts.py --interactive
```

Implementation:

```
#!/usr/bin/env python3
"""
Technical Analysis Chart Generator v1.0

Generates interactive HTML dashboard with:
- Price evolution with SMA 50/200
- Volume bars
- ADX trend strength
- ATR volatility

Features:
- Single HTML file output (no dependencies)
- Sidebar navigation with search
- 200-day lookback (configurable)
- Performant (loads all data once)
"""

import sys
import json
from pathlib import Path
from datetime import datetime, timedelta
from typing import List, Dict

import pandas as pd
import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# Project setup
project_root = Path(__file__).parent.parent
sys.path.insert(0, str(project_root))

CACHE_DIR = project_root / 'data_cache'
REPORTS_DIR = project_root / 'reports' / 'charts'
REPORTS_DIR.mkdir(parents=True, exist_ok=True)

class TechnicalChartGenerator:
    """
```

```

Generate interactive technical analysis charts
"""

def __init__(self, lookback_days: int = 200):
    self.lookback_days = lookback_days
    self.data_cache = {}

def load_indicator_data(self, symbol: str) -> pd.DataFrame:
    """
    Load indicator data for a symbol (with caching)
    """
    if symbol in self.data_cache:
        return self.data_cache[symbol]

    indicator_file = CACHE_DIR / 'indicators' / f'{symbol}_indicators.parquet'

    if not indicator_file.exists():
        print(f"Warning: No indicator data for {symbol}")
        return None

    df = pd.read_parquet(indicator_file)

    # Cache the data
    self.data_cache[symbol] = df

    return df

def create_chart(self, symbol: str, df: pd.DataFrame) -> go.Figure:
    """
    Create technical analysis chart for a symbol

    Layout:
    Row 1: Price + SMA 50/200 (main chart, 60% height)
    Row 2: Volume (20% height)
    Row 3: ADX (10% height)
    Row 4: ATR (10% height)
    """
    # Filter to lookback period
    if len(df) > self.lookback_days:
        df = df.tail(self.lookback_days)

    # Get company info
    company_info = self.get_company_info(symbol)
    name = company_info.get('name', symbol)
    sector = company_info.get('sector', 'Unknown')

    # Create subplots
    fig = make_subplots(
        rows=4, cols=1,
        shared_xaxes=True,
        vertical_spacing=0.03,
        row_heights=[0.6, 0.2, 0.1, 0.1],
        subplot_titles=(
            f'{name} ({symbol}) - {sector}',
            'Volume',
            'ADX (Trend Strength)',
            'ATR % (Volatility)'
        )
    )

    # Row 1: Price + SMAs
    fig.add_trace(
        go.Scatter(
            x=df.index,
            y=df['close'],
            name='Price',
            line=dict(color='#2E86DE', width=2),
            hovertemplate='<b>Price</b>: %{y:.2f}<br><extra></extra>'
        ),
        row=1, col=1
    )

    fig.add_trace(
        go.Scatter(
            x=df.index,
            y=df['sma_50'],
            name='SMA 50',
            line=dict(color='#FFA502', width=1.5),
            hovertemplate='<b>SMA 50</b>: %{y:.2f}<br><extra></extra>'
        ),
        row=1, col=1
    )

```

```

fig.add_trace(
    go.Scatter(
        x=df.index,
        y=df['sma_200'],
        name='SMA 200',
        line=dict(color='#FF6348', width=1.5),
        hovertemplate='<b>SMA 200</b>: %{y:.2f}<br><extra></extra>'
    ),
    row=1, col=1
)

# Row 2: Volume
fig.add_trace(
    go.Bar(
        x=df.index,
        y=df['volume'],
        name='Volume',
        marker=dict(color='#95A5A6'),
        hovertemplate='<b>Volume</b>: %{y:,.0f}<br><extra></extra>'
    ),
    row=2, col=1
)

# Row 3: ADX
fig.add_trace(
    go.Scatter(
        x=df.index,
        y=df['adx_14'],
        name='ADX',
        line=dict(color='#9B59B6', width=2),
        fill='tozeroy',
        hovertemplate='<b>ADX</b>: %{y:.1f}<br><extra></extra>'
    ),
    row=3, col=1
)

# Add ADX threshold line at 20
fig.add_hline(
    y=20,
    line_dash="dash",
    line_color="red",
    annotation_text="Threshold (20)",
    annotation_position="right",
    row=3, col=1
)

# Row 4: ATR %
fig.add_trace(
    go.Scatter(
        x=df.index,
        y=df['atr_20_pct'],
        name='ATR %',
        line=dict(color='#E67E22', width=2),
        fill='tozeroy',
        hovertemplate='<b>ATR</b>: %{y:.2f}%<br><extra></extra>'
    ),
    row=4, col=1
)

# Update layout
fig.update_xaxes(title_text="Date", row=4, col=1)
fig.update_yaxes(title_text="Price", row=1, col=1)
fig.update_yaxes(title_text="Volume", row=2, col=1)
fig.update_yaxes(title_text="ADX", row=3, col=1, range=[0, 100])
fig.update_yaxes(title_text="ATR %", row=4, col=1)

fig.update_layout(
    height=1000,
    showlegend=True,
    hovermode='x unified',
    template='plotly_white',
    legend=dict(
        orientation="h",
        yanchor="bottom",
        y=1.02,
        xanchor="right",
        x=1
    )
)

return fig

```

```
def get_company_info(self, symbol: str) -> dict:
```

```

"""Load company info from cache"""
company_file = CACHE_DIR / 'fundamentals' / 'company_info.json'

if not company_file.exists():
    return {}

with open(company_file, 'r') as f:
    company_data = json.load(f)

return company_data.get(symbol, {})

def generate_dashboard(
    self,
    symbols: List[str],
    output_file: str = None
) -> str:
    """
    Generate HTML dashboard with all charts

    Returns: Path to generated HTML file
    """
    if output_file is None:
        timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
        output_file = REPORTS_DIR / f'technical_analysis_{timestamp}.html'

    # Load all data first (for performance)
    print(f"Loading data for {len(symbols)} symbols...")
    valid_symbols = []
    for symbol in symbols:
        df = self.load_indicator_data(symbol)
        if df is not None and len(df) >= 200:
            valid_symbols.append(symbol)

    print(f"Generating charts for {len(valid_symbols)} symbols...")

    # Generate individual chart htmls
    chart_htmls = []
    for idx, symbol in enumerate(valid_symbols):
        print(f" [{idx+1}/{len(valid_symbols)}] {symbol}")
        df = self.data_cache[symbol]
        fig = self.create_chart(symbol, df)

        # Convert to HTML div (not full page)
        chart_html = fig.to_html(
            include_plotlyjs='cdn' if idx == 0 else False,
            div_id=f'chart_{symbol.replace(".", "_')}'
        )

        chart_htmls.append({
            'symbol': symbol,
            'html': chart_html
        })

    # Create master HTML with sidebar navigation
    html_content = self.create_master_html(valid_symbols, chart_htmls)

    # Write to file
    with open(output_file, 'w', encoding='utf-8') as f:
        f.write(html_content)

    print(f"\n✓ Dashboard generated: {output_file}")
    print(f"  Symbols: {len(valid_symbols)}")
    print(f"  Lookback: {self.lookback_days} days")

    return str(output_file)

def create_master_html(
    self,
    symbols: List[str],
    chart_htmls: List[Dict]
) -> str:
    """
    Create master HTML with sidebar navigation
    """
    # Build sidebar HTML
    sidebar_items = []
    for symbol in symbols:
        company_info = self.get_company_info(symbol)
        name = company_info.get('name', symbol)
        sidebar_items.append(f'''
            <div class="nav-item" onclick="showChart('{symbol.replace(".", "_')}')">
                <div class="symbol">{symbol}</div>
                <div class="name">{name}</div>
        ''')

```

```

        </div>
    '')

    sidebar_html = '\n'.join(sidebar_items)

    # Build charts HTML
    charts_html = '\n'.join([
        f'<div id="chart_container_{chart["symbol"].replace(".", "_")}" class="chart-container" style="display: none;">
{chart["html"]}</div>'
        for chart in chart_htmls
    ])

    # Complete HTML
    html = f'''
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8">
    <title>Technical Analysis Dashboard</title>
    <script src="https://cdn.plot.ly/plotly-2.18.0.min.js"></script>
    <style>
        * {{
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }}

        body {{
            font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, Oxygen, Ubuntu, Cantarell, sans-serif;
            display: flex;
            height: 100vh;
            overflow: hidden;
        }}

        #sidebar {{
            width: 300px;
            background: #2c3e50;
            color: white;
            overflow-y: auto;
            display: flex;
            flex-direction: column;
        }}

        #header {{
            padding: 20px;
            background: #34495e;
            border-bottom: 2px solid #1abc9c;
        }}

        #header h1 {{
            font-size: 20px;
            margin-bottom: 10px;
        }}

        #header p {{
            font-size: 12px;
            color: #95a5a6;
        }}

        #search {{
            padding: 15px;
            background: #34495e;
            border-bottom: 1px solid #1abc9c;
        }}

        #search input {{
            width: 100%;
            padding: 10px;
            border: none;
            border-radius: 5px;
            font-size: 14px;
        }}

        #nav {{
            flex: 1;
            overflow-y: auto;
        }}

        .nav-item {{
            padding: 15px 20px;
            cursor: pointer;
            border-bottom: 1px solid #34495e;
            transition: background 0.2s;

```

```

    }}

    .nav-item:hover {{
        background: #34495e;
    }}

    .nav-item.active {{
        background: #1abc9c;
    }}

    .nav-item .symbol {{
        font-weight: bold;
        font-size: 14px;
        margin-bottom: 5px;
    }}

    .nav-item .name {{
        font-size: 12px;
        color: #95a5a6;
    }}

    #main {{
        flex: 1;
        overflow-y: auto;
        background: #ecf0f1;
        padding: 20px;
    }}

    .chart-container {{
        background: white;
        border-radius: 8px;
        padding: 20px;
        box-shadow: 0 2px 4px rgba(0,0,0,0.1);
    }}

    #stats {{
        padding: 15px 20px;
        background: #34495e;
        border-top: 1px solid #1abc9c;
        font-size: 12px;
        color: #95a5a6;
    }}
</style>
</head>
<body>
    <div id="sidebar">
        <div id="header">
            <h1>📊 Technical Analysis</h1>
            <p>Interactive Dashboard</p>
        </div>

        <div id="search">
            <input type="text" id="searchInput" placeholder="Search symbols..." onkeyup="filterSymbols()">
        </div>

        <div id="nav">
            {sidebar_html}
        </div>

        <div id="stats">
            <div>Total Symbols: {len(symbols)}</div>
            <div>Lookback: {self.lookback_days} days</div>
            <div>Generated: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}</div>
        </div>
    </div>

    <div id="main">
        {charts_html}
    </div>

<script>
    // Show first chart by default
    const firstSymbol = '{symbols[0].replace(".", "_')}';
    showChart(firstSymbol);

    function showChart(symbolId) {{
        // Hide all charts
        const containers = document.querySelectorAll('.chart-container');
        containers.forEach(c => c.style.display = 'none');

        // Show selected chart
        const chartContainer = document.getElementById('chart_container_' + symbolId);
        if (chartContainer) {{

```

```

        chartContainer.style.display = 'block';
    })

    // Update nav selection
    const navItems = document.querySelectorAll('.nav-item');
    navItems.forEach(item => item.classList.remove('active'));

    event.currentTarget.classList.add('active');
})

function filterSymbols() {{
    const input = document.getElementById('searchInput');
    const filter = input.value.toUpperCase();
    const navItems = document.querySelectorAll('.nav-item');

    navItems.forEach(item => {{
        const symbol = item.querySelector('.symbol').textContent;
        const name = item.querySelector('.name').textContent;

        if (symbol.toUpperCase().indexOf(filter) > -1 ||
            name.toUpperCase().indexOf(filter) > -1) {{
            item.style.display = '';
        }} else {{
            item.style.display = 'none';
        }}
    }});
}}

// Keyboard navigation
document.addEventListener('keydown', function(e) {{
    const navItems = Array.from(document.querySelectorAll('.nav-item:not([style*="display: none"])'));
    const active = document.querySelector('.nav-item.active');
    const currentIndex = navItems.indexOf(active);

    if (e.key === 'ArrowDown' && currentIndex < navItems.length - 1) {{
        navItems[currentIndex + 1].click();
        navItems[currentIndex + 1].scrollIntoView({{ behavior: 'smooth', block: 'nearest' }});
    }} else if (e.key === 'ArrowUp' && currentIndex > 0) {{
        navItems[currentIndex - 1].click();
        navItems[currentIndex - 1].scrollIntoView({{ behavior: 'smooth', block: 'nearest' }});
    }}
    });
</script>
</body>
</html>
...

```

```

    return html

```

```

def main():
    """Main execution"""
    import argparse

    parser = argparse.ArgumentParser(description='Generate technical analysis charts')
    parser.add_argument('--symbols', type=str, help='Comma-separated list of symbols')
    parser.add_argument('--filter', choices=['top20', 'all'], default='top20',
                        help='Symbol filter (top20 by momentum or all qualified)')
    parser.add_argument('--lookback', type=int, default=200,
                        help='Lookback period in days')
    parser.add_argument('--output', type=str, help='Output HTML file path')
    parser.add_argument('--interactive', action='store_true',
                        help='Interactive mode (select from list)')

    args = parser.parse_args()

    # Determine symbols to chart
    if args.symbols:
        symbols = [s.strip() for s in args.symbols.split(',')]
    elif args.interactive:
        # Load qualified symbols
        qualified_file = CACHE_DIR / 'qualified' / 'qualified_symbols.json'
        with open(qualified_file, 'r') as f:
            qualified = json.load(f)

        print("\nAvailable Symbols:")
        for idx, sym in enumerate(qualified[:50], 1): # Show first 50
            print(f" {idx}. {sym['symbol']} - {sym['name']}")

        selection = input("\nEnter symbol numbers (comma-separated) or 'all': ")
        if selection.lower() == 'all':
            symbols = [s['symbol'] for s in qualified]
        else:

```

```

        indices = [int(i.strip()) - 1 for i in selection.split(',')]
        symbols = [qualified[i]['symbol'] for i in indices]
    elif args.filter == 'all':
        qualified_file = CACHE_DIR / 'qualified' / 'qualified_symbols.json'
        with open(qualified_file, 'r') as f:
            qualified = json.load(f)
        symbols = [s['symbol'] for s in qualified]
    else:
        # Top 20 by momentum
        momentum_file = CACHE_DIR / 'signals' / 'momentum_ranked.json'
        with open(momentum_file, 'r') as f:
            ranked = json.load(f)
        symbols = [s['symbol'] for s in ranked[:20]]

# Generate dashboard
generator = TechnicalChartGenerator(lookback_days=args.lookback)
output_path = generator.generate_dashboard(symbols, args.output)

print(f"\n✓ Open in browser: file://{output_path}")

if __name__ == '__main__':
    main()

```

4.2 DAILY MONITORING

Script 14: Daily Risk Monitor

File: scripts/14_daily_monitoring.py

Purpose: Check portfolio health and trigger alerts

Inputs:

- Current portfolio positions
- Latest market data (daily close)

Outputs:

- reports/daily/{YYYY-MM-DD}_monitoring.json
- Email/SMS alerts (if thresholds breached)

Execution:

```

# Run daily (automated via cron)
python scripts/14_daily_monitoring.py

```

Monitoring Checks:

```

def daily_monitoring_checks() -> dict:
    """
    Run daily portfolio health checks

    Alerts triggered if:
    1. Any position below stop-loss
    2. Portfolio drawdown > 10%
    3. Concentration > 30% (top 3 positions)
    4. Any position > 10% of account
    5. Correlation spike (top 3 > 0.85)
    6. Data staleness (no update in 2 days)
    """
    alerts = []

    # Load current state
    positions = load_portfolio_state()
    latest_data = load_latest_market_data()

    # Check 1: Stop-loss violations
    for symbol, position in positions.items():
        current_price = latest_data[symbol]['close']
        stop_price = position['current_stop_price']

        if current_price <= stop_price:
            alerts.append({
                'severity': 'HIGH',
                'type': 'stop_loss_hit',
                'symbol': symbol,
                'current_price': current_price,
                'stop_price': stop_price,
            })

```

```

        'action_required': 'Exit at next market open'
    })

# Check 2: Portfolio drawdown
peak_equity = get_peak_equity()
current_equity = calculate_current_equity()
drawdown = (current_equity / peak_equity - 1) * 100

if drawdown < -10:
    alerts.append({
        'severity': 'MEDIUM',
        'type': 'drawdown_warning',
        'current_equity': current_equity,
        'peak_equity': peak_equity,
        'drawdown_pct': drawdown,
        'action_required': 'Review risk settings'
    })

# Check 3: Concentration risk
position_values = [p['current_value'] for p in positions.values()]
position_values.sort(reverse=True)
top_3_pct = sum(position_values[:3]) / current_equity * 100

if top_3_pct > 30:
    alerts.append({
        'severity': 'LOW',
        'type': 'concentration_warning',
        'top_3_pct': top_3_pct,
        'action_required': 'Consider diversification'
    })

# ... more checks ...

return {
    'date': datetime.now().isoformat(),
    'alerts': alerts,
    'portfolio_health': 'OK' if len(alerts) == 0 else 'WARNING'
}

```

PART 5: IMPLEMENTATION ORDER

Phase 1: Data Foundation (Weeks 1-2)

```

# Week 1: Data Acquisition
scripts/01_download_eodhd_bulk.py      # EODHD bulk downloader
scripts/02_download_yahoo_fundamentals.py # Yahoo fundamentals
scripts/03_consolidate_validate_data.py # Data validator

# Week 2: Basic Analytics
scripts/04_screen_universe.py          # Universe screener
scripts/05_calculate_indicators.py      # SMA, ATR, ADX

```

Phase 2: Strategy Logic (Weeks 3-4)

```

# Week 3: Signal Generation
scripts/06_qualify_trends.py           # Trend qualifier
scripts/07_rank_momentum.py            # Momentum ranker

# Week 4: Portfolio Construction
scripts/08_calculate_position_sizes.py  # Position sizer
scripts/09_calculate_stops.py           # Stop-loss calculator
scripts/10_generate_exit_signals.py     # Exit generator

```

Phase 3: Rebalancing & Reporting (Week 5)

```

scripts/11_monthly_rebalancing.py      # Monthly rebalancer
scripts/12_generate_recommendation_report.py # Report generator

```

Phase 4: Execution & Monitoring (Week 6)

```

scripts/13_log_execution.py             # Execution logger
scripts/14_daily_monitoring.py          # Daily risk monitor
scripts/15_generate_technical_charts.py # Chart generator (NEW)

```

PART 6: CONFIGURATION FILES

config/exchanges.json

```
{
  "exchanges": {
    "NYSE": { ... },
    "NASDAQ": { ... },
    "XETRA": { ... },
    "LSE": { ... },
    "PA": { ... },
    "AS": { ... }
  }
}
```

config/filter_thresholds.json

```
{
  "NYSE": {
    "min_price_usd": 4.50,
    "min_adv_usd": 5000000,
    "min_mcap_usd": 500000000,
    "min_history_days": 252
  }
}
```

config/strategy_parameters.json

```
{
  "indicators": {
    "sma_fast": 50,
    "sma_slow": 200,
    "atr_period": 20,
    "adx_period": 14
  },
  "entry": {
    "timing": "monthly_rebalancing_date",
    "order_type": "limit",
    "limit_offset_pct": 0.5,
    "trend_qualification": {
      "sma_50_above_sma_200": true,
      "close_above_sma_50": true,
      "adx_minimum": 20
    }
  },
  "position_sizing": {
    "target_risk_per_position": 0.02,
    "min_position_pct": 0.005,
    "max_position_pct": 0.08
  },
  "stops": {
    "initial_stop_multiplier": 3.0,
    "trailing_stop_multiplier": 4.0,
    "trailing_activation_profit_pct": 0.15,
    "update_frequency": "weekly_friday"
  },
  "rebalancing": {
    "frequency": "monthly",
    "anchor": "last_trading_day",
    "execution": "first_trading_day_next_month"
  },
  "circuit_breakers": {
    "max_drawdown_pct": -15.0,
    "max_vix": 40,
    "max_correlation": 0.85,
    "max_concentration_top3_pct": 30.0,
    "max_data_staleness_days": 3
  }
}
```

APPENDIX: FILE STRUCTURE

```
trend_strategy/
├── .env                # API keys (not in git)
├──
└── config/
```

- └─ exchanges.json # Exchange definitions
- └─ filter_thresholds.json # Universe filters
- └─ strategy_parameters.json # Strategy config
- └─ scripts/
 - └─ 01_download_eodhd_bulk.py
 - └─ 02_download_yahoo_fundamentals.py
 - └─ 03_consolidate_validate_data.py
 - └─ 04_screen_universe.py
 - └─ 05_calculate_indicators.py
 - └─ 06_qualify_trends.py
 - └─ 07_rank_momentum.py
 - └─ 08_calculate_position_sizes.py
 - └─ 09_calculate_stops.py
 - └─ 10_generate_exit_signals.py
 - └─ 11_monthly_rebalancing.py
 - └─ 12_generate_recommendation_report.py
 - └─ 13_log_execution.py
 - └─ 14_daily_monitoring.py
 - └─ 15_generate_technical_charts.py # NEW
- └─ data_cache/
 - └─ raw_bulk/
 - └─ NYSE/
 - └─ 2024-01-01.parquet
 - └─ ...
 - └─ NASDAQ/
 - └─ ...
 - └─ corporate_actions/
 - └─ NYSE_splits.parquet
 - └─ NYSE_dividends.parquet
 - └─ ...
 - └─ fundamentals/
 - └─ company_info.json
 - └─ consolidated/
 - └─ AAPL.US.parquet
 - └─ MSFT.US.parquet
 - └─ ...
 - └─ indicators/
 - └─ AAPL.US_indicators.parquet
 - └─ ...
 - └─ qualified/
 - └─ qualified_symbols.json
 - └─ screening_report.csv
 - └─ signals/
 - └─ qualified_trends.json
 - └─ momentum_ranked.json
 - └─ exit_signals.json
 - └─ portfolio/
 - └─ position_sizes.json
 - └─ stop_levels.json
 - └─ metadata/
 - └─ bulk_download_log.json
 - └─ data_quality_report.json
 - └─ validation_failures.csv
- └─ data/
 - └─ portfolio_state.json # Current positions
 - └─ executions/
 - └─ {YYYY-MM-DD}_trades.json
- └─ reports/
 - └─ rebalancing/
 - └─ 2026-01_recommendations.json
 - └─ 2026-01_recommendations.pdf
 - └─ 2026-01_recommendations.csv
 - └─ daily/
 - └─ {YYYY-MM-DD}_monitoring.json
 - └─ charts/ # NEW
 - └─ technical_analysis_{timestamp}.html
- └─ logs/
 - └─ download_eodhd_{timestamp}.log

```
|— download_yahoo_{timestamp}.log
|— ...
```

SUMMARY OF CHANGES FROM v3.0 → v3.2

v3.1 Critical Fixes:

1. **Entry Signal Timing** - Now explicitly defined with monthly rebalancing anchor
2. **Circuit Breakers** - Added 5 automatic halt conditions
3. **Data Validation Thresholds** - Explicit numeric thresholds for all checks
4. **Stop-Loss Clarity** - Activation, update frequency, and ratchet mechanism explicit
5. **Rebalancing Synchronization** - Exact timing with holiday handling
6. **Chart Generator Script** - New Script 15 for technical analysis visualization

v3.2 Download Mode Simplification:

1. **3-Mode System** - Simplified from 4 modes to 3 (initial/incremental/custom)
2. **Smart Incremental Mode** - Auto-detects gap size and fills automatically
3. **New Symbol Discovery** - Automatic detection of IPOs and new listings
4. **Delisting Detection** - Flags symbols that disappear from exchanges
5. **Self-Healing** - Catches missed days automatically without manual intervention
6. **Enhanced Metadata** - Per-exchange tracking, new/delisted symbol logs

New Features:

- Circuit breaker system (prevents catastrophic losses)
- Interactive chart generator (Script 15)
- Automatic symbol discovery (new_symbols.jsonl)
- Delisting detection (delisted_symbols.jsonl)
- Smart incremental updates (single mode for all gaps)
- Data staleness monitoring
- Explicit validation thresholds

Enhanced Precision:

- Entry signals: Monthly rebalancing + top N momentum
- Stop-loss: Weekly updates on Friday close only
- Position sizing: Floor operation for whole shares
- Download modes: INITIAL (setup) → INCREMENTAL (production) → CUSTOM (manual)
- All formulas: Step-by-step with examples

Operational Simplification:

OLD (v3.0/v3.1):

```
# Multiple modes to choose from
--mode full      # Complete rebuild
--mode daily     # Yesterday only
--mode monthly   # Last 30 days
--mode custom    # Specific range
```

NEW (v3.2):

```
# Simplified to 3 modes
--mode initial    # Setup once
--mode incremental # Use for everything (auto-detects gap)
--mode custom     # Manual backfill only
```

END OF DOCUMENT

Version: 3.2

Date: February 2026

Status: Production-Ready with Critical Fixes + Smart Downloads

Next Steps: Begin Phase 1 implementation